

Automated Job & Internship Fetcher

Project Proposal — Project 2, Career Vault, ACC Portal Open Projects
Indian Institute of Technology Patna

Indranil Saha (2401CS49) | Shrut Gautam (2401CS62)

Technology requirement: React, Node.js, PostgreSQL (or another relational database).

1. Overview

Career Vault already has internship and interview experiences written by students and alumni over the years. These experiences are always about some company. Job postings are also about companies. Right now there is no connection between these two things in the portal, and this project is mainly about building that connection, not just about fetching job listings from internet.

Collecting job links from various sites is not the difficult part. The difficult part is deciding that “Google”, “Google India” and “Google LLC” are same company, finding that same internship is posted on 3 different sites, and not showing wrong information as if it is confirmed fact.

Four design points are used throughout this proposal. We are writing them here one time only, and using them again and again in later sections instead of repeating the explanation.

1.1 Cost cascade

Before doing any costly operation, first the cheaper operation is tried, so that most of the work gets removed before reaching the costly step. If every fetched page is sent directly to a language model, the running cost will become too high for a student council project to maintain after some months. This is the exact problem we want to avoid, and the order of steps described later is how it is avoided.

1.2 Loud failure

Normally when a data pipeline breaks, it does not crash with error. It silently returns zero results and the page just looks empty to the user. Nobody notices this for many days. So every automated part of this system will report its own health status, and any problem will be visible on the admin panel directly, not hidden inside some log file that nobody opens.

1.3 Additive integration

ACC Portal is already a running system with real students using it. So every database change made for this project will only add new things. Nothing already existing will be renamed, restructured or removed. (A migration means a script that changes database structure. A destructive migration is one which rewrites or deletes existing columns, and this is how a working system gets permanently broken.)

1.4 Honest data

If some value is not known, system will clearly say it is unknown. It will not put some default value which looks like real data. This point is especially important for stipend which is not disclosed, and for application deadline which is not always available. Both cases are explained in detail later in this document.

2. Integration Plan

2.1 Where it fits in the portal

The fetcher will be built as an extension inside the existing Career Vault section, not as separate application.

- Same login, same session, same navigation bar as rest of the portal.
- Same component design and visual style already used in ACC Portal.
- Two views, matching the pattern portal already follows: a public browsing view for students, and an admin review view for the limited student administrators.

No new account system is required. No second login. Roles will come from portal's existing role system itself — students will get browsing access, admins will additionally get access to review queue and source health page.

2.2 Company registry — the main connecting point

This is the most important design decision of the whole project.

The problem. Same company is written under many different names in text. One experience post may say “Google India”, one fetched job posting may say “Google LLC”, one career page may write just “google” in lower case. If the system does not resolve these, the posting and the experience will never get connected, and the whole integration idea fails.

The solution. We will keep one canonical companies table. Each row is one real organisation, and it will have a list of known aliases attached to it. Every posting and every experience will refer to a company using this ID, never using the raw text string.

Matching will be done in the following order, cheapest first:

- Exact match with known alias — free.
- Normalised match — convert to lowercase, remove legal suffixes like Ltd / Inc / Pvt, remove extra spaces — free.
- Fuzzy string matching above some similarity threshold — free.
- If still not matched, it goes to admin queue as a candidate company for manual review.

Admin merge and split option. Automatic matching will not always be correct. Sometimes 2 different companies will get merged wrongly, sometimes 1 company record will get split into two by mistake. We are stating this openly instead of claiming the matching logic is perfect, because the real difference between good and bad design here is not about how accurate the matcher is. It is about whether the mistake can be seen and corrected. So from Phase 0 itself, admin panel will have option to merge or split company records.

Once this registry is in place, following things become possible: a job posting page can show past experiences of that company, an experience page can show current open postings of that company, company pages (Section 4.1) become possible, and duplicate detection across different sources becomes reliable (Section 3.4).

2.3 Database changes

All changes will be additive only. New tables to be added:

- companies — canonical record of organisation.
- company_aliases — many aliases mapped to one company.
- postings — normalised job and internship records.
- posting_observations — first seen and last seen time, per source.

- sources — list of ingestion sources with their health status.
- extraction_reviews — admin review queue state and confidence values.

Existing experience tables will get, at most, one nullable company_id foreign key column, so that they can also connect to the company registry. No existing column will be changed or removed. Backfilling company IDs on old experience records will be a one time operation, done under admin supervision, and it can be reversed if something goes wrong.

Because postings will sit in the same PostgreSQL database as the portal's student data, eligibility filtering explained in Section 2.6 can directly use student's branch, year and CPI without copying this information anywhere else. Keeping two separate databases in sync is a known headache and we want to avoid that.

2.4 Rollout safety

Feature flag. The new Career Vault section will be behind an on/off switch which admin can toggle without doing a redeploy. If the fetcher misbehaves in front of real students, it can be turned off immediately instead of waiting for an emergency code fix.

Read only mode first. Ingestion can run and fill up the review queue before showing anything to students at all. This way the whole pipeline can be checked on real data with zero exposure to students.

Nothing gets auto-published. Every fetched posting has to go through admin review (Section 3.5) before it is visible to students. So if one extraction goes wrong, only admin's time is wasted, student's trust in the portal is not affected.

2.5 Operations view for admin

A system which runs on its own overnight needs some way to report its own status. So admin panel will get a small operations page showing:

- Last successful run of each source, with number of postings fetched.
- Which sources are currently failing or returning zero postings.
- Size of review queue, and how many items in it are low confidence.
- Extraction cost spent so far, so that cost is visible and not something discovered later on a bill.

3. Enhancements to the Listed Features

The problem statement asked for 4 things: automated fetching, integration into Career Vault, a student facing interface, and a management system. Everything written in this section is one of these 4 points, done in a better way than the obvious approach. This section is placed before Additional Features because these decisions are the base on which the later features are built.

3.1 Four tier cascade for ingestion

- **Tier 1 — ATS APIs.** Greenhouse, Lever, Ashby and Workday give public structured job board endpoints. A company using Greenhouse publishes its complete job board as JSON at a fixed URL. This is free, stable, and already in structured form, no extraction needed at all. We expect most of the volume to come from here.
- **Tier 2 — deterministic processing.** Normalisation, duplicate removal, keyword filtering, rejecting obvious junk entries. All free, no model call needed.

- **Tier 3 — Haiku extraction.** Used only for unstructured sources which did not get handled in tier 1 and 2. Converts plain text into our normalised posting schema.
- **Tier 4 — Sonnet escalation.** Used only when Haiku returns a low confidence result on some field.

The whole processing will run overnight through the Batch API, instead of running synchronously. Nothing in this workload needs to happen the same minute, and batch pricing is also cheaper for same amount of work.

3.2 Why we are not building an open web crawler

This is an intentional decision that we are not doing crawling, and we want to explain the reason instead of leaving it looking like something we forgot.

A crawler works by discovering unknown pages, following links outward from a starting point. If pointed at open internet, it will return thousands of pages daily, and most of them will not even be job postings. Checking whether a page is actually a posting or not is exactly the costly step. So the real effect of a crawler is that it increases the language model work, which is opposite of what a cost cascade is supposed to do.

Second reason is about debugging. Websites change their HTML structure without any notice. When that happens, the parser does not throw error, it simply returns nothing. Finding out why requires manually reading the new page structure and comparing with old parser logic, one site at a time, which is very slow and low value work.

What we are doing instead: a scheduled fetcher running over a fixed allowlist of sources. Same tools can be used (Crawlee, Playwright), but the approach is completely different — pulling data from known endpoints on a timer, with one adapter per source, instead of exploring the open web.

3.3 Sources, including manual input

In order of cost, cheapest first:

- ATS job board APIs.
- RSS or JSON feed, if source provides one.
- Scheduled fetcher for allowlisted career pages, one adapter per source.
- Admin manual entry.
- Student submitted link — student pastes a URL, and it goes into same review queue as automated postings.

Point 4 and 5 are not a backup plan for when automation fails. They are treated as a regular tier. Automated coverage of job market will never be 100% complete, and if we design assuming it is complete, we will silently miss many opportunities. Student submission is also useful because it turns all the students into a kind of distributed discovery network, without needing any extra infrastructure for it.

3.4 Duplicate detection without using a model

Same job role often appears on company career page, on an aggregator site, and on an ATS feed also. Student should see it only one time.

Signals used for matching, all computed without calling any model:

- Canonical company ID from the registry, not the raw company name text.
- Normalised role title — lowercase, seniority words and punctuation removed.
- Normalised location.

- Content fingerprint of the description — description text reduced to a small signature, so that near identical descriptions match even if formatting or whitespace or boilerplate text is different.

Duplicate postings get merged into one record, and this one record keeps all the source links, so student can see the posting is available on 3 different sites and choose where to apply. This is tier 2 of the cascade doing actual useful work, no LLM call needed here, and adding one would actually be a step backward, not forward.

3.5 Admin review queue with confidence score

Every extracted posting will have a numeric confidence score coming from the extraction step.

- If above threshold, it goes into normal approval queue.
- If below threshold, it goes into a separate flagged view, with the exact fields that extractor was unsure about, highlighted for admin.
- Admin can approve, edit or reject. Edits made by admin are stored, so over time we can see which type of extraction is having recurring problem.

Nothing reaches students without being reviewed. This is basically the “loud failure” point applied to ingestion side — uncertain data becomes visible work for admin, instead of becoming silently wrong data for student.

3.6 Filters, including eligibility and missing data

Students will be able to filter postings by role type, skills, location, work mode, stipend and CTC.

Eligibility filter specific to IIT Patna. Because postings are stored in same database as portal's student records, we can filter directly using branch, current year and CPI cutoff. A second year CSE student should not have to scroll past postings meant for final year mechanical students. This filter is possible only because of the shared database decision explained in Section 2.3.

Problem with compensation field. Real postings often write “competitive”, or “as per industry standard”, or a range like “■40,000–60,000 per month”, or sometimes do not mention pay at all.

If stipend is stored as single number, every posting where amount is not mentioned will silently become zero. Then a student filtering for minimum stipend will lose half the listings from result, without even knowing it happened. This failure is invisible to student and to admin both.

So stipend and CTC will be stored as 3 separate fields each: numeric minimum, numeric maximum, and a disclosure status which can be disclosed / range / not_disclosed. Filter page will have a separate checkbox option like “include postings with undisclosed compensation”, and such postings will be shown as “undisclosed” in UI, not as ■0. Same idea will be applied to any other field which is frequently left blank. Unknown is treated as a value, not as a default.

3.7 Freshness information instead of fake deadlines

Most sources do not give a reliable application deadline. Some give a deadline and then it is not accurate after some time.

From day one, posting will show something like: “First seen 6 days ago, confirmed live this morning”. Both of these are actually observed facts, coming from the posting_observations log, which records first-seen and last-seen-live time per source, on every run. Live postings will also get re-checked on a schedule, so a closed posting can be marked as expired instead of sitting there and collecting applications for no reason.

Later, once enough observation data is collected over time, same log can support typical open-duration estimate, like “postings from this company usually stay open for around 18 days”.

Two things need to be said honestly here, because overselling this feature would be worse than not building it at all. First, it will give nothing useful on day one, it needs months of data collection. Second, time between first-seen and last-seen tells us the posting's lifespan, not its actual deadline — many postings remain visible even after the position is closed. Any future estimate shown to student will be clearly labelled as an estimate, not as a confirmed deadline.

We are deliberately not showing any countdown timer. A wrong deadline is more harmful than no deadline at all, because student will trust it, deprioritise applying, and end up missing the actual deadline.

4. Additional Features

These are listed in the order of how strongly we can justify them, not in order of how new or interesting they sound.

4.1 Company pages

This is the strongest feature in this proposal, because it is both a new feature and also the proof that the integration idea from Section 2 is actually working.

One page per canonical company, containing current open postings, past internship and interview experiences already present in Career Vault, a discussion section, OA and interview pattern notes (Section 4.5), and a question bank (Section 4.6). Individual posting page will also show a small summary panel like “4 past experiences · 12 discussion replies · OA pattern available” linking to the company page.

Why company level and not posting level. First instinct is to attach discussion directly to each posting. This does not work well in practice: 300 postings will create 300 separate threads, each with maybe one comment, and discussion just dies out due to lack of concentration in one place.

There is one more structural reason also. Postings expire after some time. A discussion thread attached to posting also disappears along with it when posting closes, so same questions get asked again next year from zero. A company level page keeps growing across multiple hiring cycles and becomes more useful every year, and this is exactly the property needed for Section 4.5 and 4.6 to work well.

4.2 Resume match scoring

Student can upload resume and see a good / moderate / weak match label for each posting.

The design here is decided by simple arithmetic. 500 students multiplied by 300 live postings gives **150,000 comparisons**. Making 1 language model call per comparison completely breaks the cost model, so same 4 tier logic is applied again here:

- **Tier 1, deterministic, free.** Skill keywords are extracted once from resume at upload time, and once from each job description at ingestion time. Overlap between the two sets is just set arithmetic.
- **Tier 2, embeddings, cost is per document not per pair.** An embedding is a numeric representation of the meaning of a text, so 2 texts can be compared just by measuring distance between their number vectors. Cost is only in generating these vectors: 500 resumes plus 300 postings gives **800 embedding operations total**, and after that all 150,000 comparisons become free arithmetic between these vectors. This ratio, 800 paid operations producing 150,000 results, is the actual reason this feature stays affordable.

- **Tier 3, model call, only on demand.** When student opens a specific posting and asks why it is showing as moderate match, only then one Haiku call is made to generate the explanation text.

So the good/moderate/weak label comes fully from the free tiers, and actual written explanation comes from paid tier, and that too only when student specifically asks for it.

Handling of personal data. Resume contains name, phone number and address of student. So explicit consent will be taken at time of upload, clear statement will be shown about what data is stored and for how long, and a working delete option will be provided. This is not a minor point, since it is personal data of students being stored on an institute system.

4.3 Match aware notification

When a new posting crosses student's chosen match threshold, a notification is sent, labelled good / moderate / weak. Because these labels come from free tiers only, sending more notifications does not add per-message model cost.

Notifications will be sent as digest by default, not individually for every single posting. A feature which pings student 12 times a day will get turned off by student within one week.

4.4 Saved postings and new-since-last-visit marker

Student can save a posting and come back to it later from a Saved tab. Postings added after student's last visit will be marked as new, so a returning student can see what changed without reading whole list again. These are small features, but they are what makes a listing page worth opening more than one time.

4.5 Application tracking

From posting page itself, one click status change: Interested → Applied → In Progress → Rejected / Offer.

Kept to one click intentionally. A separate form which student has to remember to fill every time is a feature that exists in the code but not really in practice.

Aggregated and anonymised version of this data is also the most honest answer we can give to question "how many students from our campus applied here", because unlike selection statistics (Section 5.2), this data is actually generated by the portal itself.

4.6 OA and interview pattern guide

Structured summary at company level: question types, difficulty, round structure, typical timeline, all coming from submitted experiences, not made up.

Two ways this can be filled. Admin can manually write summary based on existing text experiences. Better option is a small structured form added at time of experience submission itself, so future experiences come already tagged with round type and difficulty, instead of arriving as plain text that someone has to read again to make useful. Second option is more valuable because it improves data quality right at entry point, which is the cheapest place to improve it.

4.7 Question bank

Past OA and interview questions, stored at company level, contributed by students, and reviewed by admin before publishing. Kept at company level for same reason as Section 4.1, it keeps accumulating across years, while anything attached to individual posting dies along with the posting.

Submissions must be original recollection from student's memory. Copying proprietary test content directly is not allowed, and this will be clearly stated as a submission rule, not left unwritten.

4.8 Experience weighted discussion

Inside company discussion section, a user who has an approved experience linked to that company will be ranked higher than general comments. This is only a sort key, not a full reputation system, so it is cheap to build. It solves the real problem of student forums, which is usually not shortage of content but shortage of useful signal.

5. Deferred to Later Phase

These ideas are being parked with clear reason, not silently dropped from proposal.

5.1 Alumni referral contact

Risk here is not technical. A channel which lets several hundred students message alumni for referral requests has a predictable outcome: alumni get spammed, alumni disengage from portal, and reputation damage falls on ACC, not on this feature specifically.

It also assumes existence of a verified alumni database with recorded consent for being contacted for this specific purpose, and that is a prerequisite, not something we can build as part of this project.

If built in future: should be opt-in only, no open messaging allowed, structured and rate-limited request format, and a hard cap on number of requests per student per month.

5.2 Batch selection statistics

This is a data availability problem, not an engineering problem. On-campus and off-campus selection numbers are usually held by placement cell, and are often confidential. Building UI for a table which may never get legally populated is wasted effort. So we are raising this as an open question needing answer before any design work starts. Section 4.5 gives a partial and honest substitute for this in the meantime.

5.3 Broader source coverage

More ATS providers and more allowlisted career pages can be added slowly over time. Each new source needs only one adapter plus one health check, kept deliberately small and repeatable so that growth does not require redesigning the system.

6. Delivery Phases

Phase 0 — Foundation. Company registry with admin merge/split option, normalised posting schema, source registry with health checks. Nothing else in the system works without these.

Phase 1 — Ingestion. ATS connectors, deduplication, Haiku extraction, review queue with confidence scoring.

Phase 2 — Public browsing. Listing page, eligibility and null-safe filters, posting detail page, freshness line.

Phase 3 — Linking layer. Company pages connecting postings with existing Career Vault experiences. This phase is what actually delivers on the integration claim.

Phase 4 — Personalisation. Resume upload with consent, tiered match scoring, notifications, application tracking.

Phase 5 — Community. Discussion weighting, OA pattern guide, question bank, and open-duration estimates once enough observation data is collected.

Phase 0 to Phase 3 is what we are committing to build. Phase 4 and 5 are kept as stretch goals, and we are saying this clearly now instead of promising all 13 features together and delivering only 6 of them.

7. Risks and Mitigations

Risk	Concern	Mitigation
Source breakage	Career pages change without notice and fetcher fails silently.	Per-source health check, alert on zero-result run, manual and student-submission tier keeps pipeline useful while a source is down.
Cost overrun	Extraction cost increases with volume and with how much of it is unstructured.	Cost cascade, overnight batch processing, and visible spend number in admin operations view.
Data availability	Selection statistics and alumni contact both depend on data which may not exist or may not be shareable.	Flag as open question, defer implementation instead of assuming data will be available.
Personal data	Resume is personal data being stored on institute system.	Explicit consent, stated retention period, and working deletion mechanism.

Appendix A — Posting Schema

Field	Notes
company_id	FK to canonical company. Never a raw text string.
role_title	Raw title, as published.
role_title_normalised	Used for duplicate detection.
type	internship / full-time / PPO
location	Raw and normalised value.
work_mode	onsite / hybrid / remote / unknown
skills[]	Extracted tokens, used for filtering and match tier 1.
stipend_min, stipend_max	Nullable.
stipend_disclosure	disclosed / range / not_disclosed
ctc_min, ctc_max, ctc_disclosure	Same pattern as stipend.
description_text	Full description text, used for display and for embeddings.
content_fingerprint	Signature used for duplicate detection.

source_links[]	All source links where this posting was seen.
first_seen_at	Set only once.
last_seen_live_at	Updated on every run.
status	pending_review / live / expired / rejected
extraction_confidence	Numeric value, decides flagged queue.
deadline_stated	Nullable, filled only if source actually publishes a deadline.

The disclosure columns and the nullable deadline field are where the honest-data principle stops being just an intention and becomes an actual constraint enforced by the schema. Interface cannot show a misleading zero value, because schema itself does not store one.

8. Conclusion

Main goal of this project is to make Career Vault more useful, without turning ACC Portal into a second separate job website. Job postings alone are commodity information, a student can find them somewhere else also. But postings connected to interview experiences, OA patterns and questions already written by students of this campus are not available anywhere else, and that connection is the actual thing worth building here.

First release will focus on company registry, reliable ingestion, duplicate handling, admin review, student browsing with eligibility filters, and company pages connecting postings to existing experiences. Personalisation and community features can be added in later phases, once this foundation is working properly.